GRAPH
PROBLEM 2

QUESTION:

DS	**Session**	Graph	**Question Information**	Level 1 Challenge 82

Question description

Udaya family is going to order a large pizza and eat it together. A total of n family members will join the order, and there are m possible toppings. The pizza may have any number of toppings.

Each family member gives two wishes concerning the toppings of the pizza. The wishes are of the form "topping x is good/bad". Your task is to choose the toppings so that at least one wish from everybody becomes true (a good topping is included in the pizza or a bad topping is not included).

Input

The first input line has two integers n and m: the number of family members and toppings. The toppings are numbered 1,2,...,m.

After this, there are n lines describing the wishes. Each line has two wishes of the form "+ x" (topping x is good) or "- x" (topping x is bad).

Output

Print a line with m symbols: for each topping "+" if it is included and "-" if it is not included. You can print any valid solution.

If there are no valid solutions, print "IMPOSSIBLE".

Constraints

- 1≤n,m≤10^5
- 1≤x≤m

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 5
+ 1 + 2
- 1 + 3
+ 4 - 2
```

EXPECTED OUTPUT

```
- + + + -
```

Test Case 2

INPUT (STDIN)

```
3 5
+ 2 + 2
- 3 + 3
+ 5 - 2
```

EXPECTED OUTPUT

```
- + - - +
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void link(int i,int j)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
600
```

Test Case 3

NLOC

```
71
```

SOURCE CODE:

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_LITERALS 200005

typedef struct EdgeItem {
    int dest;
    struct EdgeItem* next;
} EdgeItem;

EdgeItem* fwd_head[MAX_LITERALS];
EdgeItem* rev_head[MAX_LITERALS];

void append_implication(int src, int dst) {
    EdgeItem* e1 = (EdgeItem*)malloc(sizeof(EdgeItem));
    e1->dest = dst;
    e1->next = fwd_head[src];
    fwd_head[src] = e1;

    EdgeItem* e2 = (EdgeItem*)malloc(sizeof(EdgeItem));
    e2->dest = src;
    e2->next = rev_head[dst];
    rev_head[dst] = e2;
}

int finish_stack[MAX_LITERALS], stack_top = 0;
bool seen[MAX_LITERALS];
int component_id[MAX_LITERALS], num_scc = 0;

void pass_dfs(int u) {
    seen[u] = true;
    for (EdgeItem* cur = fwd_head[u]; cur != NULL; cur = cur->next) {
        if (!seen[cur->dest]) pass_dfs(cur->dest);
    }
    finish_stack[stack_top++] = u;
}

void scc_dfs(int u, int c) {
    component_id[u] = c;
    for (EdgeItem* cur = rev_head[u]; cur != NULL; cur = cur->next) {
        if (component_id[cur->dest] == 0) scc_dfs(cur->dest, c);
    }
}

int main() {
    int voters, toppings;
    if (scanf("%d %d", &voters, &toppings) != 2) return 0;

    for (int i = 0; i < voters; i++) {
        char s1[4], s2[4];
        int t1, t2;
        scanf("%s %d %s %d", s1, &t1, s2, &t2);
        int lit_a = (s1[0] == '+') ? (2 * t1) : (2 * t1 + 1);
        int lit_b = (s2[0] == '+') ? (2 * t2) : (2 * t2 + 1);
        append_implication(lit_a ^ 1, lit_b);
        append_implication(lit_b ^ 1, lit_a);
    }

    int max_lit = 2 * toppings + 1;
    for (int lit = 2; lit <= max_lit; lit++) {
        if (!seen[lit]) pass_dfs(lit);
    }
    for (int i = stack_top - 1; i >= 0; i--) {
        int lit = finish_stack[i];
        if (component_id[lit] == 0) scc_dfs(lit, ++num_scc);
    }

    for (int t = 1; t <= toppings; t++) {
        if (component_id[2 * t] == component_id[2 * t + 1]) {
            printf("IMPOSSIBLE\n");
            return 0;
        }
    }

    for (int t = 1; t <= toppings; t++) {
        char decision = (component_id[2 * t] > component_id[2 * t + 1]) ? '+' : '-';
        printf("%c%s", decision, (t == toppings) ? "" : " ");
    }
    printf("\n");
    return 0;
}
```

OUTPUT:

```
3 5
+ 1 + 2
- 1 + 3
+ 4 - 2
- + + + -
```

```
3 5
+ 2 + 2
- 3 + 3
+ 5 - 2
- + - - +
```